Support Vector Machine (SVM) Classifier

1. Introduction to SVM

What is SVM?

Support Vector Machine (SVM) is a supervised machine learning algorithm used mainly for classification. It can also be used for regression.

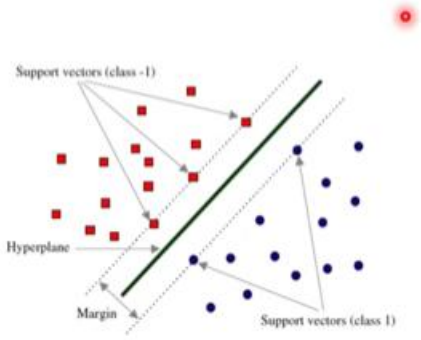
Because SVM is supervised, it learns from a labeled dataset containing input features and their corresponding target labels.

Example: Classifying patients into two categories:

- Class 0: No diabetes
- Class 1: Diabetes

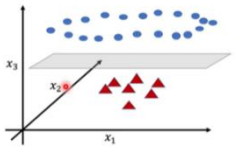
SVM learns a decision boundary from training data and uses it to classify unseen examples.

Important terminology



1. Hyperplane

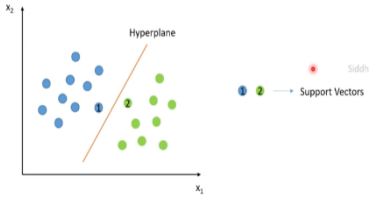
The decision boundary that separates different classes. In two dimensions, it is a line; in three dimensions, it is a plane; in higher dimensions, it is called a hyperplane.



SVM in 3 dimensions

2. Support vectors

The training data points closest to the decision boundary. These points are particularly important in determining the position of the optimal hyperplane. If these data points change, the position of the hyperplane changes.

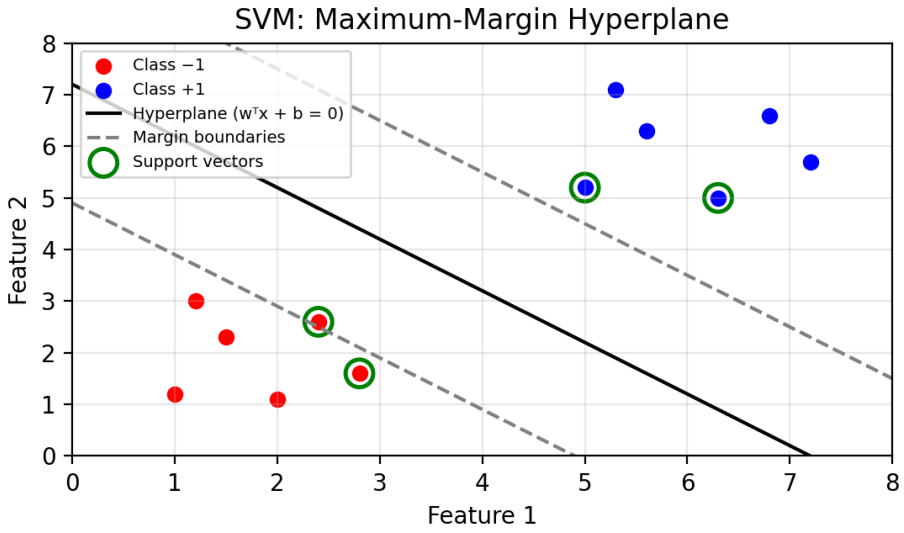


3. Margin

The distance between the boundaries on either side of the separating hyperplane. SVM aims to maximize this margin while managing classification errors.

Why is the margin important?

Imagine two lines can separate the same two classes. One passes very close to some training points, while the other leaves a larger gap between the classes. SVM generally prefers the boundary with the larger margin because it provides a wider separation between the classes.



Advantages and disadvantages

Advantages	Disadvantages
Effective for small and medium-sized datasets	Training can be slow on very large datasets
Works well with high-dimensional data	Sensitive to feature scaling
Can model nonlinear decision boundaries using kernels	Noisy data and overlapping classes can make classification difficult
Uses support vectors to define the boundary	Kernel and regularization choices can require tuning

Important: SVM does not require every training point to be correctly classified. Soft-margin SVM allows some violations to improve generalization, clear margin of separation works efficiently.

2. Mathematical Foundations of SVM

2.1 The hyperplane equation

For a dataset with n input features, the decision boundary is defined by:

$$\mathbf{w^T x + b = 0}$$

Where:

- x = input feature vector
- w = weight vector
- b = bias
- $w^T x$ = dot product of weights and features

For two features, the equation becomes:

$$\mathbf{w_1x_1 + w_2x_2 + b = 0}$$

This equation represents a straight line in two-dimensional space.

2.2 Classification rule

SVM calculates a decision score:

$$\mathbf{f(x) = w^T x + b}$$

The predicted class depends on the sign of this score:

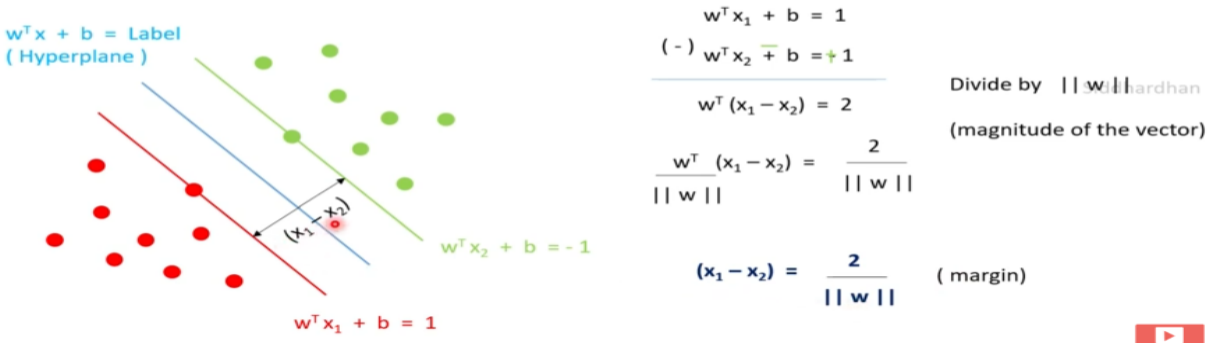
$$\mathbf{\hat{y} = +1 \text{ if } f(x) > 0;}$$

$$\mathbf{\hat{y} = -1 \text{ if } f(x) < 0}$$

If the score is zero, the point lies exactly on the decision boundary.

SVM commonly uses labels −1 and +1 during mathematical optimization. If the original dataset uses labels 0 and 1, predictions can be converted back afterward.

2.3 How does SVM maximize the margin?



For a linearly separable dataset, SVM seeks a hyperplane that separates the classes with the maximum margin.

The two margin boundaries are:

$$w^T x + b = +1$$

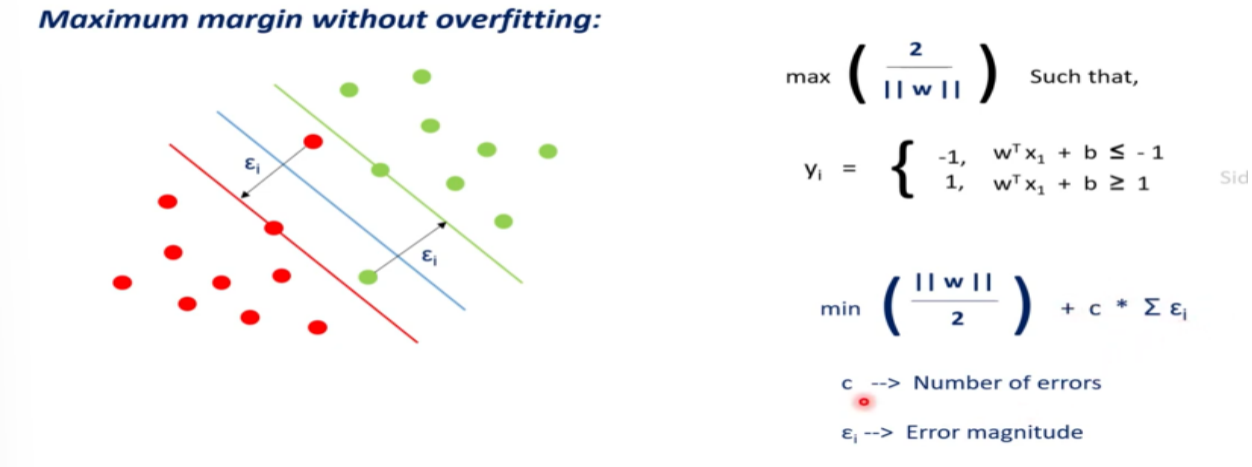
$$w^T x + b = -1$$

The distance between these boundaries is:

$$\text{Margin} = 2/(\|w\|)$$

Here, ‖w‖ is the Euclidean norm of the weight vector.

Therefore, maximizing the margin is equivalent to minimizing the norm of the weight vector.



In the standard hard-margin formulation, the optimization problem is:

$$\text{min_}(w,b) = (1/2)\ \|w\|$$

subject to:

$$y_i(w^T x_i + b) \geq 1$$

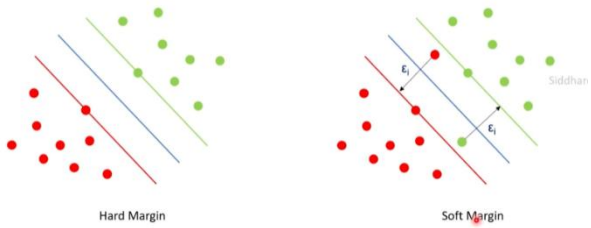
for every training example i, where $y_i \in \{-1, +1\}$.

This constraint ensures that every training point is correctly classified and lies outside or on its required margin boundary.

2.4 What if the data is not linearly separable?

Real-world data often contains overlapping classes, noise, and nonlinear relationships. Two concepts help SVM handle these problems:

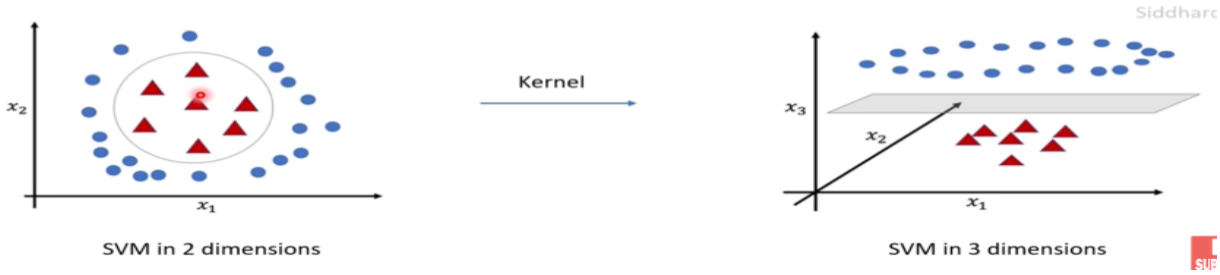
- **Soft margin:** Allows some points to violate the margin or be misclassified.
- **Kernel trick:** Allows SVM to learn nonlinear decision boundaries.



These are related but different techniques: soft margins handle violations, while kernels help model nonlinear separation.

3. Kernels in SVM

Kernel Function generally transforms the training set of data so that a non-linear decision surface can be transformed to a linear equation in a higher number of dimension spaces. It returns the inner product between two points in a standard feature dimension.



3.1 Why do we need kernels?

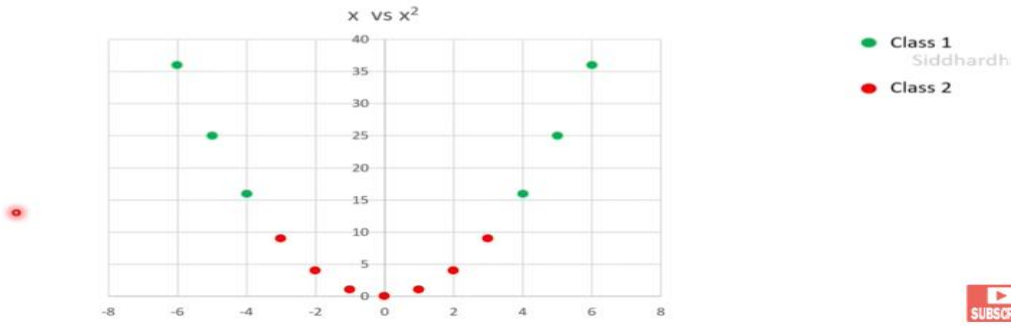
Consider a dataset where one class forms a circle surrounded by another class.

Feature (x)	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6



A single straight line cannot separate these classes correctly. A nonlinear decision boundary is needed.

Feature (x)	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6
x^2	36	25	16	9	4	1	0	1	4	9	16	25	36



The kernel trick enables SVM to learn such boundaries by evaluating relationships between points as if they had been represented in a higher-dimensional feature space, without explicitly calculating every transformed feature.

3.2 Common kernel types

1. Linear kernel

Suitable when classes can be separated approximately by a straight line or hyperplane.

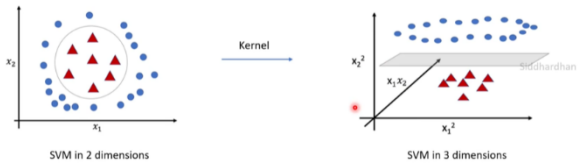
$$K(x,z) = x^T z$$

2. Polynomial kernel

Can model interactions and curved relationships using polynomial feature combinations.

$$K(x,z) = (\gamma x^T z + r)^d$$

Here, d is the polynomial degree.



3. Radial Basis Function (RBF)/(Gaussian) kernel

A flexible choice for nonlinear relationships. It measures similarity based on the distance between data points.

$$K(x,z) = e^{(-\gamma\|x - z\|^2)}$$

4. Sigmoid kernel

Uses a sigmoid-shaped similarity function:

$$K(x,z) = \tanh(\gamma x^T z + r)$$

It is less commonly used than the linear and RBF kernels in typical SVM applications.

3.3 When should you choose each kernel?

Dataset characteristics	Initial choice
Approximately linear separation	Linear
Polynomial-like relationships	Polynomial
Complex nonlinear patterns	RBF
Specific reason to use a sigmoid similarity function	Sigmoid

Start with a linear kernel for a simple baseline. Try RBF when the relationship appears nonlinear, then compare models using validation data.

Note: Kernel performance depends on the dataset and hyperparameter settings. A more complex kernel does not automatically produce better accuracy.

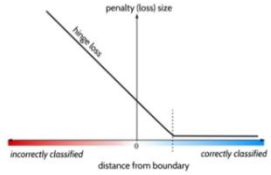
4. Hinge Loss

4.1 What is hinge loss?

Hinge loss is a loss function commonly used to train SVM classifiers. It penalizes incorrect predictions and points that fall inside the required margin.

$L = \max(0, 1 - y_i (w^T x_i + b))$

0 - for correct classification
1 - for wrong classification



For a single example:

$$L_i = \max(0, 1 - y_i f(x_i))$$

Where:

- $y_i \in \{-1, +1\}$ is the true label.
- $f(x_i) = w^T x_i + b$ is the decision score.
- L_i is the hinge loss for that example.

4.2 Understanding hinge loss

Misclassification :

$y_i = 1 \quad \hat{y}_i = -1$

$L = (1 - (1)(-1)$

$L = (1 + 1)$

$L = 2$ (High loss Value)

$y_i = -1 \quad \hat{y}_i = 1$

$L = (1 - (-1)(1)$

$L = (1 + 1)$

$L = 2$ (High loss Value)

$L = \max(0, 1 - y_i (w^T x_i + b))$

0 - for correct classification
1 - for wrong classification

Correct classification :

$y_i = 1 \quad \hat{y}_i = 1$

$L = (0 - (1)(1)$

$L = (0 - 1)$

$L = -1$ (Low loss Value)

$y_i = -1 \quad \hat{y}_i = -1$

$L = (0 - (-1)(-1)$

$L = (0 - 1)$

$L = -1$ (Low loss Value)

Condition	Interpretation	Hinge loss
$y_i f(x_i) \geq 1$	Correctly classified and outside or on the margin	0
$0 < y_i f(x_i) < 1$	Correctly classified but inside the margin	Between 0 and 1
$y_i f(x_i) \leq 0$	Misclassified or on the decision boundary	At least 1

For example, suppose $y_i = +1$.

- If $f(x_i) = 2$, then $L_i = \max(0,1 - 2) = 0$.
- If $f(x_i) = 0.4$, then $L_i = \max(0,1 - 0.4) = 0.6$.
- If $f(x_i) = -0.5$, then $L_i = \max(0,1 - (-0.5)) = 1.5$.

The first example has no hinge loss because it is correctly classified with sufficient margin.

4.3 Hinge loss versus binary cross-entropy

Hinge loss	Binary cross-entropy
Commonly used for SVM classification	Commonly used for logistic regression
Uses labels −1, + 1 in its standard formulation	Usually uses labels 0,1
Penalizes violations of a margin	Penalizes incorrect probability estimates
Does not directly produce probabilities	Works with predicted probabilities

5. Soft-Margin SVM and Regularization

A hard-margin SVM assumes that the data can be separated perfectly. In real-world datasets, this assumption is often unrealistic.

Soft-margin SVM introduces slack variables ξ_i that permit margin violations.

Its standard primal optimization objective is:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m}\xi_i$$

subject to:

$$y_i(w^T x_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0$$

Here:

- ξ_i measures the margin violation for example i.
- C controls the trade-off between maximizing the margin and penalizing violations.
- m is the number of training examples.

A large C penalizes violations more strongly, while a smaller C permits more violations in exchange for potentially better generalization.

A note on lambda (λ)

Some from-scratch implementations use λ as the regularization coefficient. For example, an objective may be written as:

$$J(w) = \frac{\lambda}{2}\|w\|^2 + \frac{1}{m}\sum_{i=1}^{m} L_i$$

This is a regularized hinge-loss formulation. The precise relationship between λ and C depends on the objective's scaling and convention; they should not automatically be treated as identical numerical values.

6. Gradient Descent for SVM

Gradient-based optimization adjusts the weights and bias to reduce the objective function.

The general update rule is:

$$w_{(new)} = w_{(old)} - \eta \, \partial J/\partial w$$

$$b_{(new)} = b_{(old)} - \eta \, \partial J/\partial b$$

Where η is the learning rate.

For a common regularized hinge-loss objective, define the margin condition:

$$y_i(w^T x_i + b) \geq 1$$

There are two cases.

Case 1: The point satisfies the margin condition

If $y_i(w^T x_i + b) \geq 1$, the hinge-loss gradient for this example is zero. The regularization term can still contribute to the weight update.

if $(y_i \,.\, (w.x + b) \geq 1)$:

$$\frac{dJ}{dw} = 2\lambda w$$

$$\frac{dJ}{db} = 0$$

else $(y_i \,.\, (w.x + b) < 1)$:

$$\frac{dJ}{dw} = 2\lambda w - y_i \,.\, x_i$$

$$\frac{dJ}{db} = y_i$$

Case 2: The point violates the margin condition

If $y_i(w^T x_i + b) < 1$, the hinge-loss gradient contributes to updating both the weights and bias.

$$w_2 = w_1 - L * \frac{dJ}{dw}$$

$$b_2 = b_1 - L * \frac{dJ}{db}$$

For the average hinge-loss objective

$$J(w,b) = \lambda/2 \, \|w\|^2 + (1/m) \, \Sigma(i = 1 \text{ to } m) \max(0, 1 - y_i(w^T x_i + b)),$$

the gradients, away from the hinge's nondifferentiable point, are:

$$\nabla_w J = \lambda w - (1/m) \, \Sigma(i: y_i f(x_i) < 1) \, y_i x_i$$

$$\partial J/\partial b = -(1/m) \, \Sigma(i: y_i f(x_i) < 1) \, y_i$$

These equations show why the model updates differently depending on whether a point meets its margin requirement.

Important: The exact gradient equations depend on how the loss and regularization terms are defined. Some tutorials use summed losses rather than average losses, which changes the scaling.

7. Implementing SVM from Scratch Using NumPy

The following implementation illustrates the mathematical approach from the lessons. It uses a linear SVM trained with full-batch gradient descent and regularized hinge loss.

It is an educational implementation, not a complete replacement for optimized SVM libraries.

7.1 Class structure

Function	Purpose
`__init__()`	Stores the learning rate, iteration count, and regularization coefficient

Function	Purpose
`fit()`	Initializes parameters and trains the model
`update_weights()`	Calculates gradients and updates weights and bias
`predict()`	Computes scores and converts them into class predictions

7.2 Complete NumPy implementation

```
import numpy as np


class SVM:
    def __init__(self, learning_rate=0.001,
                 n_iters=1000, lambda_param=0.01):
        self.lr = learning_rate
        self.n_iters = n_iters
        self.lambda_param = lambda_param
        self.w = None
        self.b = None

    def fit(self, X, y):
        X = np.asarray(X, dtype=float)
        y = np.asarray(y)

        # This implementation expects binary labels 0 and 1.
        classes = np.unique(y)
        if len(classes) != 2 or not np.array_equal(classes, [0, 1]):
            raise ValueError("y must contain both 0 and 1.")

        # Convert labels from {0, 1} to {-1, +1}.
        y_encoded = np.where(y == 1, 1, -1)

        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        self.b = 0.0

        for _ in range(self.n_iters):
            self.update_weights(X, y_encoded)

        return self

    def update_weights(self, X, y):
        n_samples = X.shape[0]

        # Find points that violate the margin.
        margins = y * (X @ self.w + self.b)
        violating = margins < 1

        # Gradient of regularization plus average hinge loss.
        dw = self.lambda_param * self.w
        db = 0.0

        if np.any(violating):
            dw -= np.sum(
```

```
                y[violating, None] * X[violating],
                axis=0
            ) / n_samples

            db -= np.sum(y[violating]) / n_samples

        # Gradient descent update.
        self.w -= self.lr * dw
        self.b -= self.lr * db

    def predict(self, X):
        X = np.asarray(X, dtype=float)
        scores = X @ self.w + self.b

        # Convert {-1, +1} predictions back to {0, 1}.
        return (scores >= 0).astype(int)
```

7.3 How the code works

1. **Initialization:** `w` and `b` start at zero.
2. **Label encoding:** The original labels `0` and `1` are converted to `-1` and `+1`.
3. **Margin calculation:** The model computes $y_i(w^T x_i + b)$.
4. **Loss condition:** Points with margin below 1 contribute to hinge loss.
5. **Parameter update:** The weights and bias are updated using gradient descent.
6. **Prediction:** A nonnegative score maps to class 1; a negative score maps to class 0.

Note that the implementation uses an average hinge-loss convention. Other from-scratch implementations may use different scaling or update equations.

8. Applying SVM to a Diabetes Dataset

The lesson describes applying the custom SVM class to a diabetes dataset using Pandas and scikit-learn preprocessing tools.

Workflow

1. Load `diabetes.csv` using Pandas.
2. Separate input features (`X`) and target labels (`y`).
3. Split the data into training and test sets.
4. Standardize the features.
5. Train the custom SVM model.
6. Predict training and test labels.
7. Evaluate performance.

Example training and evaluation code

This example assumes that the `SVM` class above has been saved in `svm.py` and that `diabetes.csv` is available.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
```

```
from svm import SVM

# 1. Load dataset
df = pd.read_csv("diabetes.csv")

# 2. Separate features and target
X = df.drop(columns=["Outcome"]).to_numpy()
y = df["Outcome"].to_numpy()

# 3. Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 4. Standardize without leaking test information
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 5. Train the custom model
model = SVM(
    learning_rate=0.001,
    n_iters=1000,
    lambda_param=0.01
)
model.fit(X_train, y_train)

# 6. Evaluate
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("Training accuracy:",
      accuracy_score(y_train, train_pred))
print("Test accuracy:",
      accuracy_score(y_test, test_pred))

print(classification_report(y_test, test_pred))
```

Why standardization matters

SVM uses dot products and distances, so features with large numerical ranges can dominate the optimization. Standardization places features on comparable scales.

The scaler must be fitted only on the training data. Applying `fit_transform()` to the entire dataset before splitting can leak information from the test set.

Results reported by the lesson

The source summary reports approximately:

- Training accuracy: **77%**
- Test accuracy: **75%**

These are the results reported in the source summary, not guaranteed outputs from the code above. Actual performance depends on the implementation, dataset, split, and hyperparameters.

Research note: Diabetes classification is a medical prediction task. Accuracy alone does not establish that a model is safe or suitable for clinical use.

Machine Learning Interview Questions 2

Q1. What is unsupervised learning?

Unsupervised learning **uses data without target labels**. The algorithm attempts to discover patterns, groups, or relationships within the data.

For example, a business may group customers based on purchasing behavior without having predefined customer categories.

The lesson focuses on two types.

1 Clustering

Clustering **groups similar data points together**.

Example: Grouping customers into clusters based on purchasing behavior.

Common algorithms include:

- K-Means
- Hierarchical Clustering
- DBSCAN

2 Association

Association learning **discovers relationships between items or events**.

Example: Customers who buy bread may frequently buy butter as well.

A common technique is the Apriori algorithm, often used in market-basket analysis.

Clustering	Association
Groups similar data points	Finds relationships among items
Produces clusters	Produces association rules
Example: customer segmentation	Example: product recommendation patterns

Q2. Classification vs. Regression

Both are common supervised learning tasks, but they predict different kinds of outputs.

Feature	Classification	Regression
Output	A category or class	A continuous numerical value
Example	Diabetes: positive or negative	House price prediction
Algorithms	Logistic Regression, SVM, Decision Tree	Linear Regression, Regression Tree
Common metrics	Accuracy, precision, recall, F1-score	MAE, MSE, RMSE, R^2

Interview answer: Classification predicts a discrete class label, whereas regression predicts a numerical value.

Q3. Dimensionality Reduction

What is dimensionality reduction?

Dimensionality reduction **decreases the number of features used to represent a dataset** while attempting **to preserve important information.**

For example, a dataset containing 100 features might be transformed into a smaller representation containing 10 components.

Potential benefits include:

- Lower computational cost.
- Easier visualization.
- Reduced redundancy.
- Potentially improved model generalization.

Two common techniques

1. Feature selection

Selects **a subset of the original features** which contribute most to predict the output**.**

Example: Keeping 10 relevant features from a dataset of 100 features.

2. Feature extraction

Creates new features by transforming or **combining the original features**.

Example: PCA transforms correlated features into principal components.

Feature selection	Feature extraction
Keeps selected original features	Creates transformed features
Original meaning is often easier to retain	New features may be harder to interpret

Feature selection	Feature extraction
Example: SelectKBest	Example: PCA

Q4. Why Do We Evaluate Machine Learning Models?

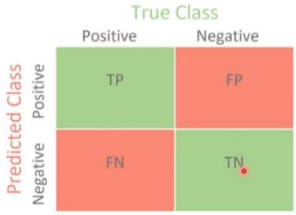
Training performance alone **does not tell us how well a model will perform on unseen data**.

Evaluation helps us:

1. Estimate generalization performance.
2. Compare competing models.
3. Identify overfitting or underfitting.
4. Understand different types of prediction errors.
5. Choose a model appropriate for the application.

A test set should remain separate from the data used to train the model and tune its hyperparameters.

Q5. Classification Evaluation Metrics



1. Confusion matrix

A confusion matrix compares actual labels with predicted labels.

For binary classification:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- **TP:** The model correctly predicts a positive case.
- **TN:** The model correctly predicts a negative case.
- **FP:** The model predicts positive when the actual class is negative.
- **FN:** The model predicts negative when the actual class is positive.

2. Important classification metrics

Accuracy

The fraction of all predictions that are correct.

$$\textbf{Accuracy} = (TP + TN)/(TP + TN + FP + FN)$$

Precision

Of the cases predicted positive, how many are actually positive?

$$\textbf{Precision} = TP/(TP + FP)$$

Recall

Of all actual positive cases, how many did the model identify?

$$\textbf{Recall} = TP/(TP + FN)$$

F1-score

The harmonic mean of precision and recall.

$$F1 = 2\,(Precision \times Recall)/(Precision + Recall)$$

Example

Suppose a classifier produces:

- TP = 40
- TN = 50
- FP = 10
- FN = 20

Then:

$$\textbf{Accuracy} = (40 + 50)/120 = 75\%$$

$$\textbf{Precision} = 40/(40 + 10) = 80\%$$

$$\textbf{Recall} = 40/(40 + 20) \approx 66.7\%$$

$$F1 \approx 72.7\%$$

This shows why accuracy alone can be insufficient. The model has 75% accuracy, but it misses approximately one-third of the actual positive cases.

For an imbalanced dataset, precision, recall, F1-score, and the confusion matrix can provide useful additional information.

Q6. Regression Evaluation Metrics

Regression metrics measure the difference between predicted numerical values and actual values.

Let y_i be the actual value, $\hat{y}_i$ the predicted value, and m the number of observations.

1. Mean Absolute Error (MAE)

$$MAE = (1/m)\,\Sigma(i = 1 \text{ to } m)\,|y_i - \hat{y}_i|$$

MAE measures the average absolute prediction error. It is expressed in the same units as the target.

2. Mean Squared Error (MSE)

$$MSE = (1/m)\,\Sigma(i = 1 \text{ to } m)\,(y_i - \hat{y}_i)^2$$

MSE squares the errors, so larger errors receive disproportionately greater penalties.

3. Root Mean Squared Error (RMSE)

$$\mathbf{RMSE = \sqrt{MSE}}$$

RMSE returns the error to the target's original units while retaining a stronger penalty for large errors than MAE.

4. Coefficient of determination (R^2)

$$\mathbf{R^2 = 1 - (\Sigma(i)\ (y_i - \hat{y}_i)^2)/(\Sigma(i)\ (y_i - \bar{y})^2)}$$

Here, $\bar{y}$ is the mean of the actual target values.

R^2 compares the model's squared prediction error with the error from predicting the target mean. A higher value generally indicates better fit, but R^2 can be negative on test data.

Metric	What it measures	Preferred direction
MAE	Average absolute error	Lower
MSE	Average squared error	Lower
RMSE	Square root of average squared error	Lower
R^2	Fit relative to a mean-prediction baseline	Generally higher

Final Revision Sheet

Concept	Remember this
SVM	Supervised algorithm that aims for a maximum-margin decision boundary
Hyperplane	Decision boundary defined by $w^T x + b = 0$
Support vectors	Training points that constrain the learned boundary
Margin	Standard margin width is $2/\|\|w\|\|$
Kernel trick	Enables nonlinear decision boundaries without explicitly computing every transformed feature
Hinge loss	$\max(0,1 - y_i f(x_i))$
Soft margin	Permits margin violations to handle imperfectly separable data
C	Controls the penalty for violations in a common SVM formulation
Gradient descent	Updates model parameters using gradients
Standardization	Helps prevent large-scale features from dominating SVM training
Unsupervised learning	Learns patterns without target labels
Clustering	Groups similar data points
Association	Finds relationships among items
Dimensionality reduction	Reduces the number of features or dimensions
Confusion matrix	Summarizes classification outcomes
MAE, MSE, RMSE, R^2	Common regression evaluation metrics

Self-Test Questions

Try answering these without looking at the notes.

1. Why does SVM try to maximize the margin?
2. What is the difference between a support vector and an ordinary training point?
3. Why do we use kernels?
4. When does hinge loss become zero?
5. What is the difference between hard-margin and soft-margin SVM?
6. Why must we fit `StandardScaler` on the training data only?
7. What is the difference between clustering and association?
8. Why can accuracy be misleading for an imbalanced dataset?
9. What is the difference between feature selection and feature extraction?
10. Which regression metric penalizes large errors more strongly: MAE or MSE?